Background

The company dataset I will explore in this project is from PriceRunner. PriceRunner is an independent price comparison service that helps shoppers find better products and cheaper prices in order to make smart decisions and save money. It was founded in 1999 and is active in Sweden, the UK, Denmark, and Norway, however, it also includes shipping from foreign countries. They have over 5.8 million products and 15.4 million visits every month in all countries. The dataset was collected from the UC Irvine Machine Learning. UCI Machine Learning Repository is a collection of databases, data generators, and domain theories wherein it is widely used by the machine learning community, students, researchers, and educators for the analysis of ML algorithms and gathering machine learning datasets.

The dataset that will be used in my project is called *Product Classification and Clustering*, which contains 35,311 product offers or instances and consists of 10 categories, provided by 306 different merchants. According to Akritidis (2020), this dataset is suitable for clustering, classification, entity-matching algorithms, and problems involving text-mining, in the context of product data. The dataset is in a tabular and text format and it shows that there was previous data preprocessing performed such as case folding and punctuation removal on the titles of column 2. It is stated under the variables table that there are no missing values on all 7 features in the dataset. The variables include Product ID, Product Title, Merchant ID, Cluster ID, Cluster Label, Category ID, and Category Label.

Sources: About PriceRunner. (2022) [Online] PriceRunner United Kingdom. Available from: https://www.pricerunner.com/info/about-pricerunner
Akritidis, L. (2020). Product Classification and Clustering [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5M91Z.
UCI Machine Learning Repository. (2019) [Online] Uci.edu. Available from: https://archive.ics.uci.edu/

Data Preparation

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn import datasets, metrics
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, accuracy_score
from sklearn.model_selection import cross_val_score
from sklearn.metrics import confusion_matrix
from sklearn.cluster import KMeans
from collections import Counter
import re
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import cross_val_score, StratifiedKFold
from imblearn.over_sampling import SMOTE
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.metrics import confusion_matrix, roc_auc_score, roc_curve,
classification_report, accuracy_score, ConfusionMatrixDisplay
from scipy.stats import ks_2samp
```

The first and foremost important step in every data analysis process is to important libraries like Seaborn, Pandas, Numpy, and Matplotlib, which are generally used for data manipulation, visualization, general plotting, and numerical computation, Additionally, libraries like 'sklearn', involves machine learning algorithms like preprocessing, metrics, cluster, KNeighborsClassifier, RandomForestClassifier, and more. These libraries are crucial for exploratory data analysis, classification, and clustering.

```
df = pd.read_csv("C:\\Users\\amand\\Downloads\\YEAR 2\\Term 2\\pricerunner_aggregate.csv")
```

Executing the 'pd.read_csv' function of Pandas will allow the program to read the PriceRunner dataset or the CSV file itself.

```
df.shape

(35311, 7)
```

The '.shape' attribute of Numpy shows the number of rows and columns respectively. It displays that there are 35,311 rows and 7 columns in total.

	Product ID	Product Title	Merchant ID	Cluster ID	Cluster Label	Category ID	Category Label
0	1	apple iphone 8 plus 64gb silver	1	1	Apple iPhone 8 Plus 64GB	2612	Mobile Phones
1	2	apple iphone 8 plus 64 gb spacegrau	2	1	Apple iPhone 8 Plus 64GB	2612	Mobile Phones
2	3	apple mq8n2b/a iphone 8 plus 64gb 5.5 12mp sim...	3	1	Apple iPhone 8 Plus 64GB	2612	Mobile Phones
3	4	apple iphone 8 plus 64gb space grey	4	1	Apple iPhone 8 Plus 64GB	2612	Mobile Phones
4	5	apple iphone 8 plus gold 5.5 64gb 4g unlocked ...	5	1	Apple iPhone 8 Plus 64GB	2612	Mobile Phones

The '.head()' method of Pandas displays the top 5 rows of the dataset, which provides a quick glance of the dataset. It is observed that each row represents a product offering, along with its merchant provider and the category it belongs to. In this table, it is seen that the dataset includes mobile phones and possibly other types of consumer electronics. Each product offer is assigned a unique 'Product ID', while 'Merchant ID', 'Cluster ID', and 'Category ID' depend on the details and grouping of the product. I can consider this dataset to be product-related data that can be extracted for further data analysis.

```
df.dtypes
```

```
Product ID          int64
Product Title      object
 Merchant ID        int64
 Cluster ID         int64
 Cluster Label     object
 Category ID        int64
 Category Label    object
dtype: object
```

With the '.dtypes' object of Numpy, I can identify the type of data each column accepts. The columns 'Product ID', 'Merchant ID', 'Cluster ID', and 'Category ID' are int64 data types which mean it stores the whole numbers. Meanwhile, 'Product Title', 'Cluster Label', and 'Category Label' is object type that deals with string or mixed data types.

```
df.info()
```

The '.info' function of Pandas lays out a summary for each column in the DataFrame, including its column name, non-null count, data type, and the total number of entries. There are a total of 35,311 entries, with an index starting at 0. It is observed that all of the columns do not have any missing values, which makes it advantageous for data analysis and eliminates the need for extensive data cleaning.

```
df.describe(include='all').T
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
Product ID	35311.0	NaN	NaN	NaN	26150.800176	13498.19122	1.0	14958.5	27614.0	37508.5	47358.0
Product Title	35311	30993	washing machine	90	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Merchant ID	35311.0	NaN	NaN	NaN	120.501883	117.045557	1.0	17.0	75.0	253.0	371.0
Cluster ID	35311.0	NaN	NaN	NaN	30110.687633	18410.265642	1.0	6123.0	40630.0	44059.0	47525.0
Cluster Label	35311	12849	Canon IXUS 185	27	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Category ID	35311.0	NaN	NaN	NaN	2618.14293	3.600708	2612.0	2615.0	2619.0	2622.0	2623.0
Category Label	35311	10	Fridge Freezers	5501	NaN	NaN	NaN	NaN	NaN	NaN	NaN

The '.describe' method is used for providing summary statistics of the DataFrame. It reveals that there are 30,993 unique product titles, with "washing machine" being the most frequent (appeared 90 times). The Product ID would not be relevant to the analysis since it's a unique assignment for each product offering. The maximum value of Merchant ID is 371, which means the IDs range from 1 to 371, with each product associated with a merchant. The ClusterID ranges from 1 to 47,525. On the other hand, there are 12,849 unique Cluster Label, with Canon IXUS 185 being the most frequent label, which appeared 27 times. The CategoryID ranges closely from 2,612 to 2,623, which means there are few categories but they cover a vast number of product offerings. That said, the Fridge Freezers is the most frequent category (appeared 5,501 times) in the Category Label.

```
df.isnull().sum()
```

```
Product ID         0
Product Title      0
 Merchant ID       0
 Cluster ID        0
 Cluster Label     0
 Category ID       0
 Category Label    0
dtype: int64
```

Again, this confirms that none of the columns have missing values.

```python
# Strip unnecessary spaces from column names
df.columns = df.columns.str.strip()
# Verify changes
print(df.columns)
```

```
Index(['Product ID', 'Product Title', 'Merchant ID', 'Cluster ID',
       'Cluster Label', 'Category ID', 'Category Label'],
      dtype='object')
```

In this code block, I removed unnecessary space at the beginning of some of the columns through the .strip() method to avoid random errors from white spaces in the later code.

```python
# Check for duplicates
duplicates = df.duplicated().sum()
print(f"\nNumber of duplicate rows: {duplicates}")
```

```
Number of duplicate rows: 0
```

I can use the 'duplicated()' and '.sum()' methods to check the total number of duplicated rows. In this case, I don't have any duplicated rows. This means that the PriceRunner dataset already has a solid foundation for data analysis.

```python
# Check unique values
for i in df.columns:
    print(i,':',df[i].nunique())
```

```
Product ID : 35311
Product Title : 30993
Merchant ID : 306
Cluster ID : 13233
Cluster Label : 12849
Category ID : 10
Category Label : 10
```

This code block displays the number of unique values in each column. Based on the results, there are 306 unique merchants. There are also 10 unique 'Category ID', which match the number of unique 'Category Label'.

```
# Enumerate unique Category Labels
category_label_mapping = {label: idx for idx, label in enumerate(df['Category
Label'].unique())}

# Display the mapping
print(category_label_mapping)
```

```
{'Mobile Phones': 0, 'TVs': 1, 'CPUs': 2, 'Digital Cameras':
3, 'Microwaves': 4, 'Dishwashers': 5, 'Washing Machines': 6,
'Freezers': 7, 'Fridge Freezers': 8, 'Fridges': 9}
```

To further expound on the 'Category Labels' using mapping and dictionary, I can observe that the categories consist of Mobile Phones, TVs, CPUs, Digital Cameras, Microwaves, Dishwashers, Washing Machines, Freezers, Fridge Freezers, and Fridge, wherein the product offerings are grouped into one of these categories.

```
# Check for categorical features
categorical_cols = df.select_dtypes(include=['object']).columns
print("\nCategorical Columns:")
print(categorical_cols)

# Check for numerical features
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
print("\nNumerical Columns:")
print(numerical_cols)
```

```
Categorical Columns:
Index(['Product Title', 'Cluster Label', 'Category Label'], dt
ype='object')

Numerical Columns:
Index(['Product ID', 'Merchant ID', 'Cluster ID', 'Category I
D'], dtype='object')
```

In this code block, I can see which features belong to categorical and numerical columns. This is crucial for preprocessing data and feature selection when used for analyses and models later on.

Before I proceeded with the data visualization, I had a deep observation and cross-checking when I got these results from this code:

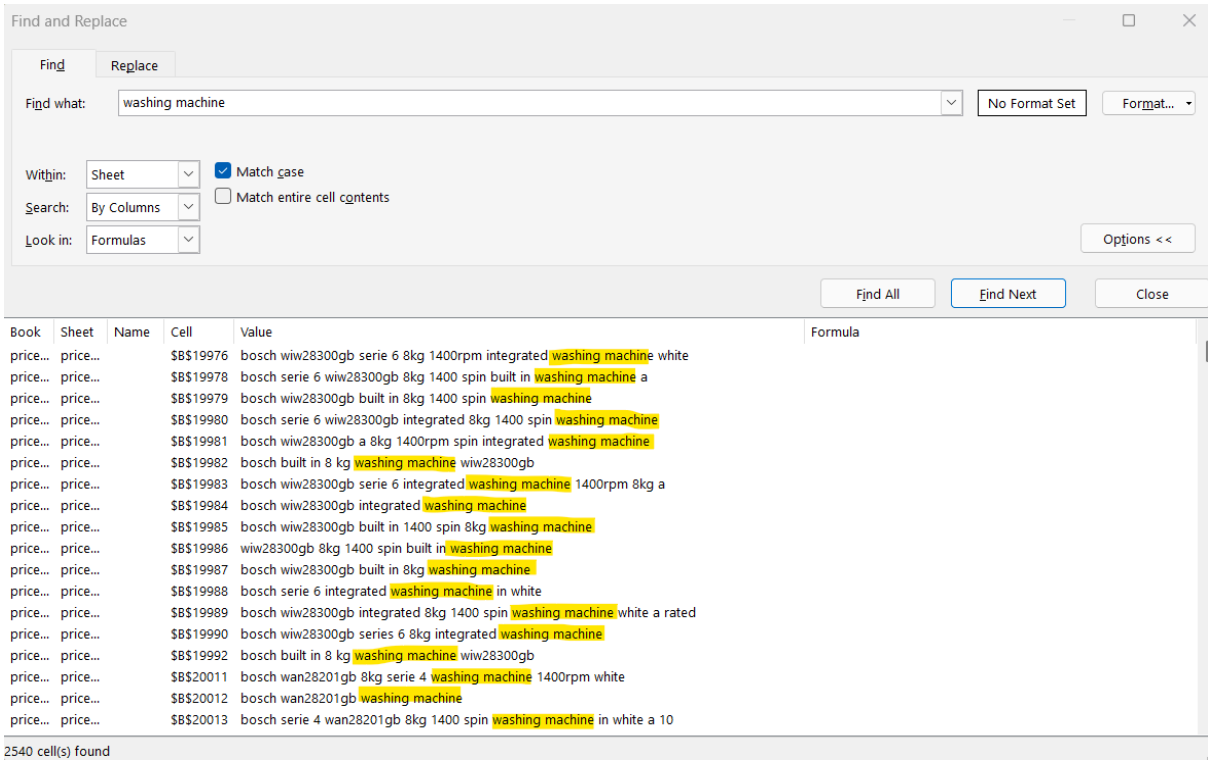
```
# Frequency counts for categorical columns
for col in categorical_cols:
    print(f"\nTop 5 most frequent values:")
    print(df[col].value_counts().head())
```

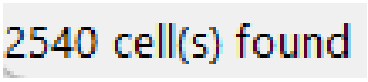
```
Top 5 most frequent values:
Product Title
washing machine                          90
built in fully integrated dishwasher     35
frost free fridge freezer                34
washer dryer                             24
american fridge freezer                  22
Name: count, dtype: int64

Top 5 most frequent values:
Cluster Label
Canon IXUS 185              27
Canon PowerShot SX730 HS    24
Samsung UE49NU7100          24
Apple iPhone 8 Plus 64GB    23
Samsung UE65NU7100          23
Name: count, dtype: int64

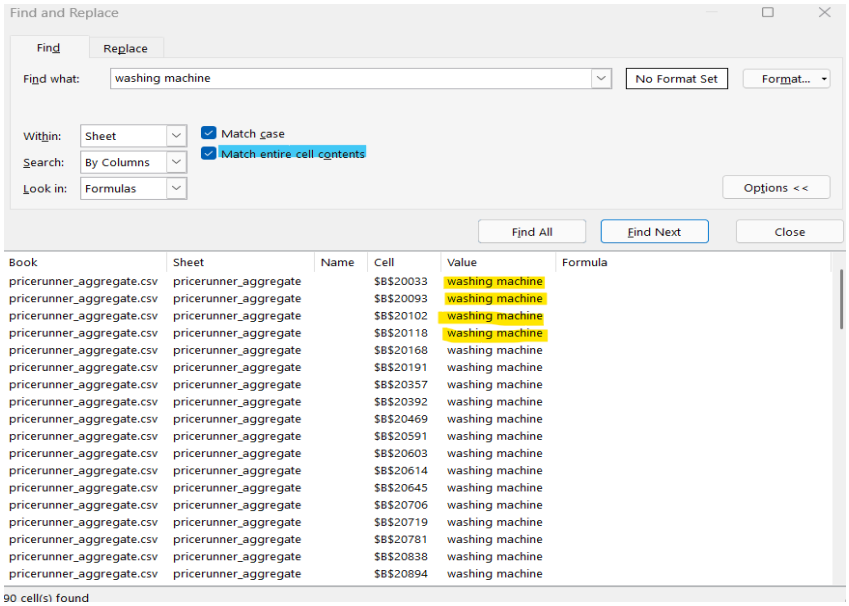
Top 5 most frequent values:
Category Label
Fridge Freezers     5501
Mobile Phones       4081
Washing Machines    4044
CPUs                3862
Fridges             3584
Name: count, dtype: int64
```

I noticed that the most frequent values of product titles have a very low value (90,35,34,24,22) despite having more than 35,000+ entries, which seems doubtful. With that, I cross-checked in the CSV file through Excel if the most frequent values were really “washing machine” and the succeeding product titles.





In Excel, the results present that 2,540 entries on the dataset have “washing machine” on them as Product Title values, which proves the frequent values I got from Jupyter Notebook were not accurate (90 values). I then found out that the 90 values result was because the program matched the entire cell contents with the word “washing machine” so only entries with the word exactly “washing machine” on them are counted. The entry “washing machine” is treated differently from a longer description entry like “bosch wan28201gb washing machine”.





With that realization, I looked back again at the format of the Product Title values and I observed that there is a noticeable inconsistency. There are seen to be extra details, misspelled words, and ordering of words. There is no standardization on the values as I examine through, and it is better to properly identify them through Category Label or Cluster Label instead when doing analysis.

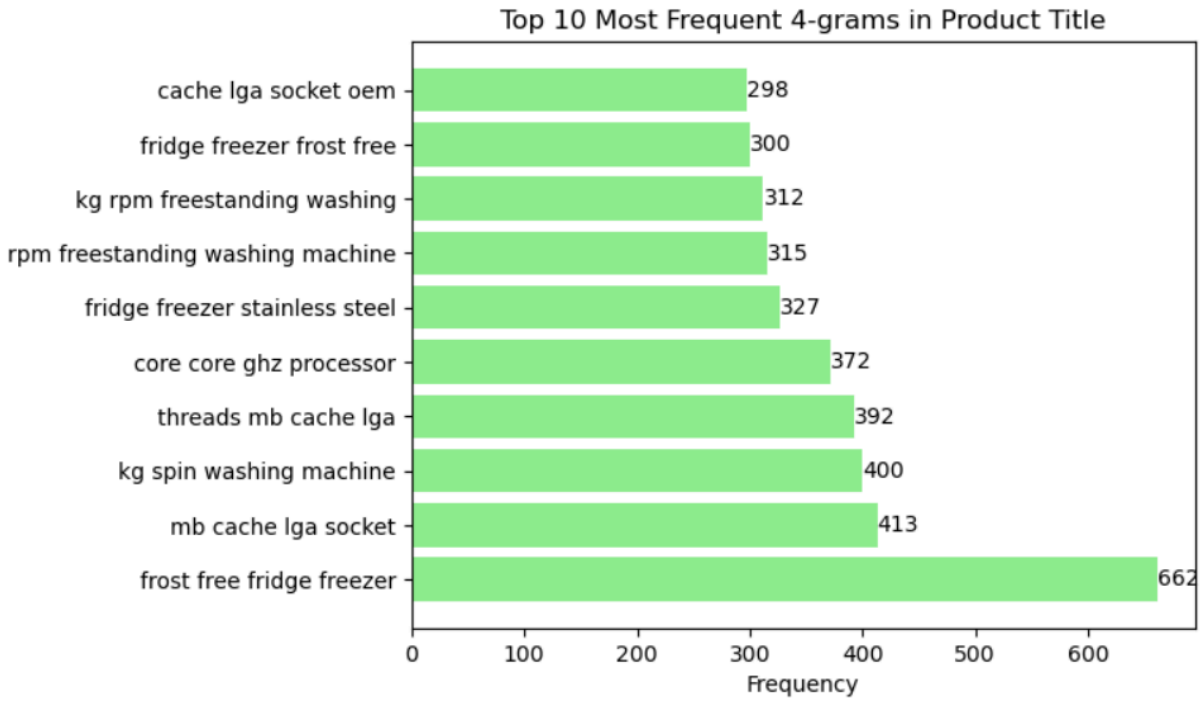
That being said, I created a 4-gram visualization instead to capture multi-word patterns to make a more accurate representation since it considers the combination of words rather than isolated words for full context. It aggregates relevant keyword clusters rather than overfitting to generic words like “washing machine”. This makes much more sense than the previous results.

Exploratory Data Analysis

For a better analysis of EDA, I formulated a main business question:
How can we improve product categorization, merchant performance, and overall sales strategy based on product titles, categories, clusters, and merchant behavior?

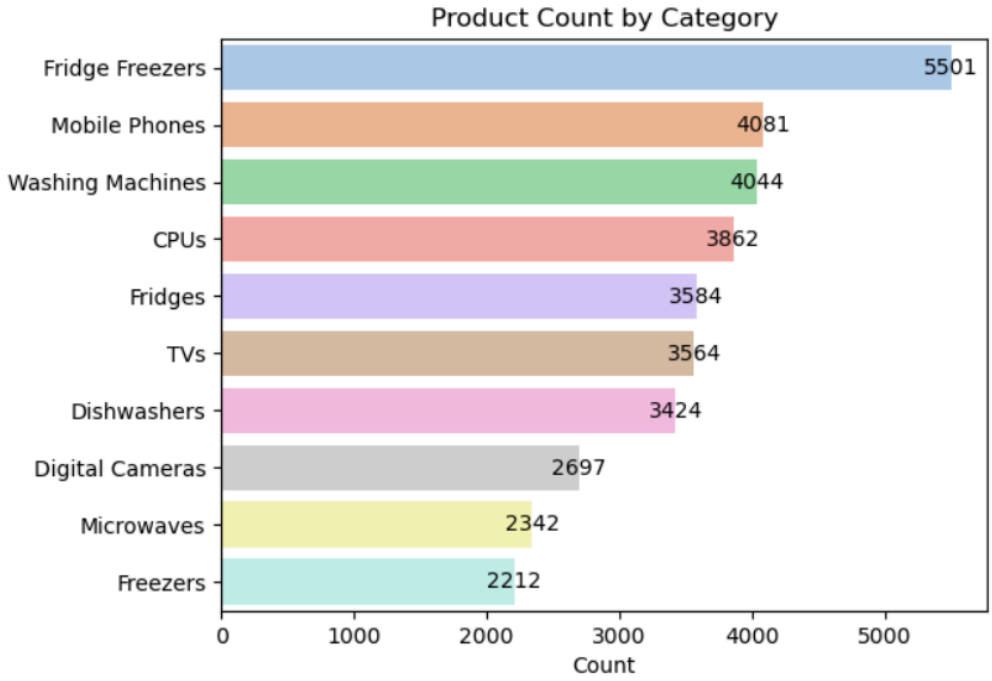
I also formulated supporting questions to answer the main question at the conclusion:

Visualisation #1: What are the most common 4-word combinations in product titles, and how can they help improve product categorization for the business?



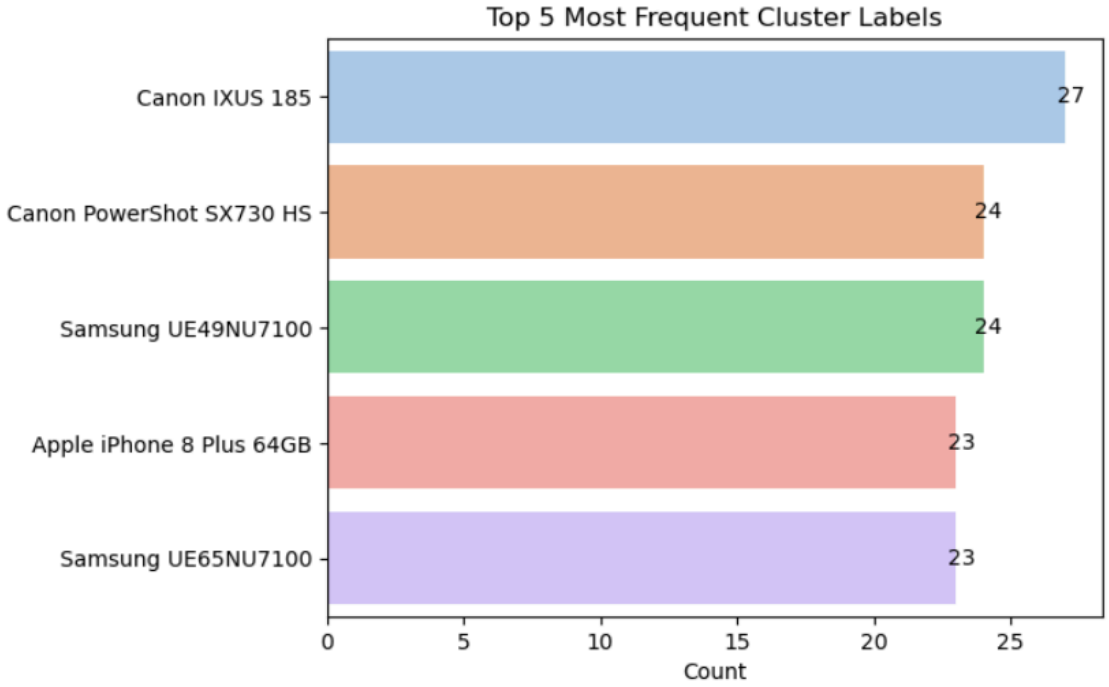
It is evident here that “frost-free fridge freezer” appears most frequently in the dataset, with 662 occurrences. Next are “mb cache lga socket”, “kg spin washing machine”, “threads mb cache lga”, and “core ghz processor”, wherein the counts are closer to each other. 4-grams can enhance product categorization, especially in identifying important product attributes. I used 4-grams in product titles because it can help me identify the most frequent combinations of terms related to particular product features or types which makes it easier to search. This is more effective than using value.counts() since it provides inaccurate results. I want to determine which product offers occur most frequently in the dataset using keywords. Aside from categorization, 4-grams improves the searchability of the products in the dataset. Overall, this method can assist in grouping relevant products together or facilitate product bundling to encourage upselling.

Visualisation #2: Which product categories have the most products, and how can we use this for optimization?



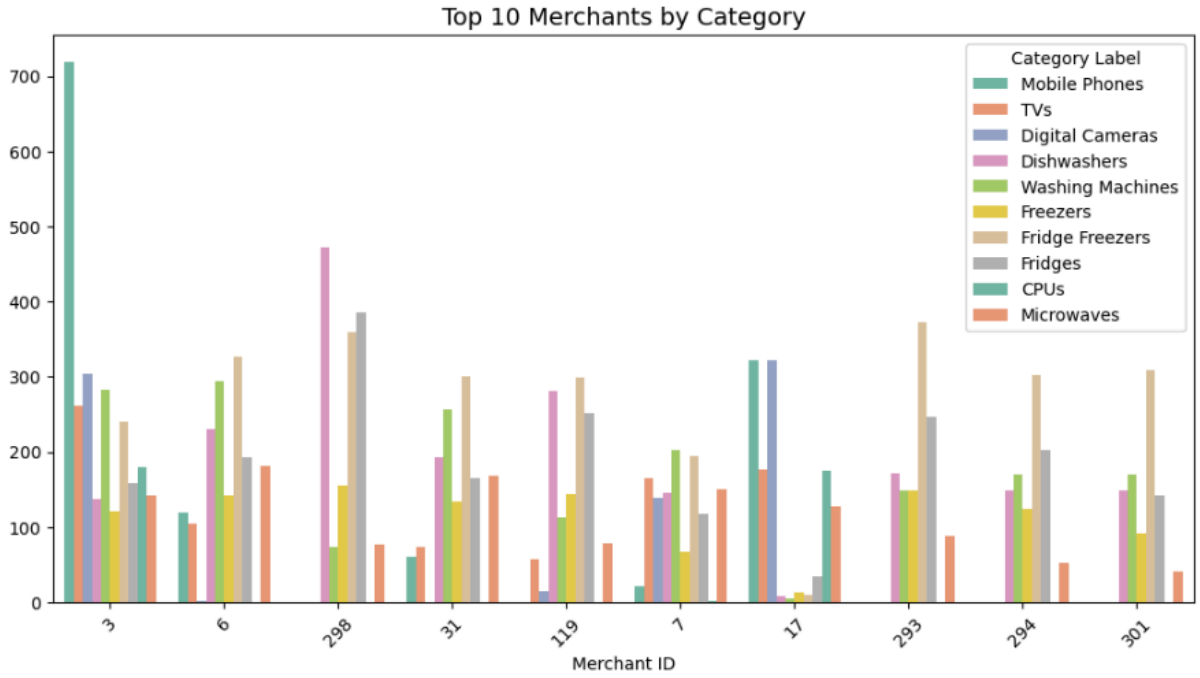
The product category with the highest number of products is Fridge Freezers, with a value of 5,501. It has a noticeable difference of more than 1,000 products compared to categories like Mobile Phones, Washing Machines, CPUs, Fridges, etc… The large number of fridge freezer products available implies that the market may be oversaturated, which leads to intense competition with many options for customers to choose from. Therefore, the company can focus on advertising and promoting the top categories, and highlighting these categories depending on the season or demand. The company can create great campaigns for this category like “Top Fridge Freezers of 2025” or “Popular Mobile Phones under £1000”. Since most of these are high-end products, it is best to improve the comparison features into a more detailed one wherein there are user reviews, ratings, and side-by-side comparisons of specifications. With that, customers can better navigate and understand the different product offerings in the same category.

Visualisation #3: What are the most frequent product clusters, and how can clustering boost sales?



The most frequent product cluster is the Canon IXUS 185, with 27 occurrences, followed by the Canon PowerShot, Samsung UE49NU7100, Apple iPhone 8 Plus, and Samsung UE65NU7100, respectively. Canon models show high clustering frequencies, while Samsung TV models appear in two variants. Clustering is useful for product grouping because the company can put clustered products together in a bundle to create great deals for customers and increase sales. Clusters significantly help in grouping products by their brand, model, features, and price points. It can also improve the recommendation systems of the website by suggesting products within the same cluster to customers based on their searches, browsing, and purchase history. Since Canon IXUS 185, the company can design targeted marketing campaigns like advertising a cluster of Canon products with advertisements that showcase professional use features. Overall, this will enhance customers' shopping experience.

Visualisation #4: How do merchants perform across different categories, and how can this information optimize performance?



Top 10 Merchants with the Most Products:

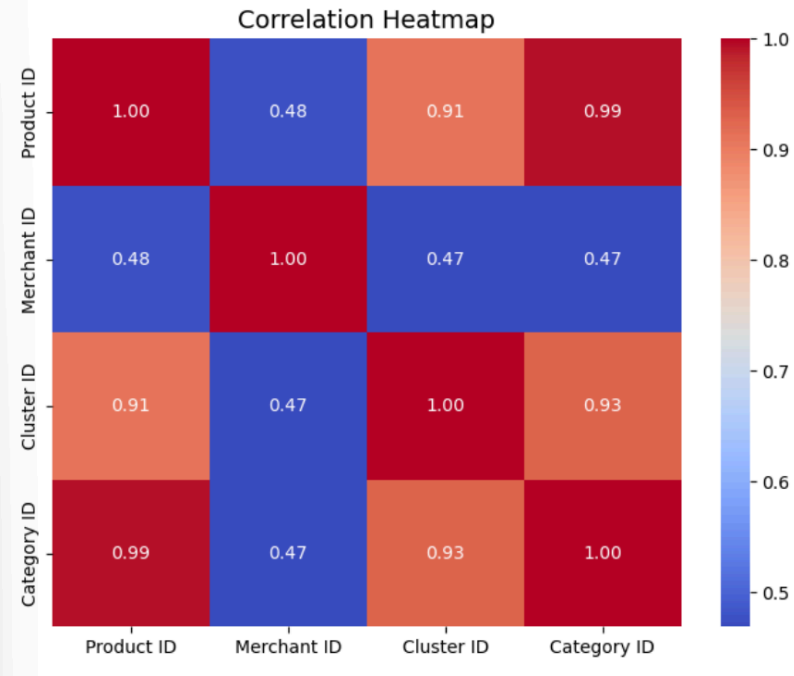
[91]:

```
Merchant ID
3      2547
6      1591
298    1523
31     1350
119    1239
7      1204
17     1193
293    1177
294    1000
301     901
Name: count, dtype: int64
```

It is evident that the top-performing merchant is Merchant #3, with a total of 2,547 products. #3 stands out by concentrating on Mobile Phones, with over 700 products, while also offering products from different categories, which shows his diversity. Aside from #3, Merchant #298 is particularly strong in Dishwashers, with over 450 products. I noticed that Fridge Freezers, Freezers, and TVs have a consistent presence across several merchants. Additionally, #17 offers much more for Mobile Phones and Digital Cameras, which focuses on a tech-savvy customer base. The performance can be optimized when merchants focus on their strongest categories and continue to build inventory in these areas. The company can build strong relationships with top merchants and offer them better promotional support and marketing efforts.

PriceRunner can also optimize the website interface to make it easy for the shopper to find and compare products of merchants like intuitive navigation and filter options.

Visualisation #5: Which variables show strong relationships with each other?



It is observed that Product ID and Category ID have a very strong correlation, nearly perfect at 0.99. This suggests that products are nearly aligned with their categories which implies systematic product categorization. Cluster ID and Category ID place the second highest positive correlation, which indicates that clusters are greatly aligned with the product categories. This means that the clustering algorithms successfully group similar products. Lastly, the correlation of 0.91 between Cluster ID and Product ID shows that effective product clustering is based on similar characteristics.

Conclusion:

4-grams is useful for product titles because it effectively determines frequent combinations of terms, which helps highlight significant product attributes and categorization accurately. This method also improves upselling and product bundling by grouping relevant products together. Systematic product categorization is seen with a strong correlation between Category ID and Product ID. Merchants must focus on categories they highly concentrate on, while also maintaining an optimal inventory for each item. It is important to build strong relationships with top-performing merchants and provide them with better promotional support. Along with that, the company can enhance product comparison by having detailed review ratings and side-by-side

comparisons so customers make better shopping decisions. Clustering enhances the recommendation systems and optimizes the website interface for intuitive navigation.

Machine Learning

Classification

My target variable is Category Label. After doing the necessary preparation in preparing train and test data, the accuracy and classification report of the models were too high and some had perfect scores (100% accuracy). This means that the model predicted a category correctly all the time. Although it may not be bad in some cases (data is well-prepared, few noise elements…), it may also indicate that the model is overfitting, imbalanced, or easy-to-predict data. Therefore, it is very important to verify if the model is accurate.

```
Evaluating Random Forest Model
Accuracy of Random Forest: 100.00%
                  precision    recall  f1-score   support

            CPUs       1.00      1.00      1.00       778
 Digital Cameras       1.00      1.00      1.00       537
     Dishwashers       1.00      1.00      1.00       690
        Freezers       1.00      1.00      1.00       444
 Fridge Freezers       1.00      1.00      1.00      1081
         Fridges       1.00      1.00      1.00       693
      Microwaves       1.00      1.00      1.00       472
   Mobile Phones       1.00      1.00      1.00       859
             TVs       1.00      1.00      1.00       729
Washing Machines       1.00      1.00      1.00       780

        accuracy                           1.00      7063
       macro avg       1.00      1.00      1.00      7063
    weighted avg       1.00      1.00      1.00      7063

Evaluating Decision Tree Model
Accuracy of Decision Tree: 100.00%
                  precision    recall  f1-score   support

            CPUs       1.00      1.00      1.00       778
 Digital Cameras       1.00      1.00      1.00       537
     Dishwashers       1.00      1.00      1.00       690
        Freezers       1.00      1.00      1.00       444
 Fridge Freezers       1.00      1.00      1.00      1081
         Fridges       1.00      1.00      1.00       693
      Microwaves       1.00      1.00      1.00       472
   Mobile Phones       1.00      1.00      1.00       859
             TVs       1.00      1.00      1.00       729
Washing Machines       1.00      1.00      1.00       780

        accuracy                           1.00      7063
       macro avg       1.00      1.00      1.00      7063
    weighted avg       1.00      1.00      1.00      7063
```

```
Evaluating K-Nearest Neighbors Model
Accuracy of K-Nearest Neighbors: 99.86%
                  precision    recall  f1-score   support

            CPUs       1.00      1.00      1.00       778
 Digital Cameras       1.00      1.00      1.00       537
     Dishwashers       1.00      1.00      1.00       690
        Freezers       1.00      0.99      0.99       444
 Fridge Freezers       1.00      1.00      1.00      1081
         Fridges       1.00      1.00      1.00       693
      Microwaves       1.00      1.00      1.00       472
   Mobile Phones       1.00      1.00      1.00       859
             TVs       1.00      1.00      1.00       729
Washing Machines       1.00      1.00      1.00       780

        accuracy                           1.00      7063
       macro avg       1.00      1.00      1.00      7063
    weighted avg       1.00      1.00      1.00      7063

Evaluating Logistic Regression Model
Accuracy of Logistic Regression: 100.00%
                  precision    recall  f1-score   support

            CPUs       1.00      1.00      1.00       778
 Digital Cameras       1.00      1.00      1.00       537
     Dishwashers       1.00      1.00      1.00       690
        Freezers       1.00      1.00      1.00       444
 Fridge Freezers       1.00      1.00      1.00      1081
         Fridges       1.00      1.00      1.00       693
      Microwaves       1.00      1.00      1.00       472
   Mobile Phones       1.00      1.00      1.00       859
             TVs       1.00      1.00      1.00       729
Washing Machines       1.00      1.00      1.00       780

        accuracy                           1.00      7063
       macro avg       1.00      1.00      1.00      7063
    weighted avg       1.00      1.00      1.00      7063
```

I also did cross-validation to evaluate the model on different splits of the data. However, the models still performed exceptionally well with high accuracy scores:

```
Evaluating Random Forest Model with Cross-Validation
Cross-Validation Accuracy of Random Forest: 100.00%
Cross-Validation Scores: [100. 100. 100. 100. 100.]
Evaluating Decision Tree Model with Cross-Validation
Cross-Validation Accuracy of Decision Tree: 100.00%
Cross-Validation Scores: [100. 100. 100. 100. 100.]
Evaluating K-Nearest Neighbors Model with Cross-Validation
Cross-Validation Accuracy of K-Nearest Neighbors: 99.90%
Cross-Validation Scores: [99.86425339 99.90950226 99.88687783 99.89819005 99.95475113]
Evaluating Logistic Regression Model with Cross-Validation
Cross-Validation Accuracy of Logistic Regression: 100.00%
Cross-Validation Scores: [100. 100. 100. 100. 100.]
```

After doing extensive trial and error with the model, I finally identified and solved the problem. To address the issue of very high accuracy and to improve the model's performance, I configured the model's settings which made the results more realistic. I identified that the model's problems stemmed from class/data imbalance and mostly overfitting.

Starting off with the Random Forest model, I reduced:

- n_estimators from 50 to 5
- Max_depth from 10 to 3 and
- Min_samples_split was set to 10 (by default, it is 2)

Reducing the n_estimators means that the model has fewer trees to learn from, previously there were too many trees. The max_depth prevents it from overfitting to the data and makes the model more generalized. The min_samples_split was increased to lower the likelihood of overfitting the small details in the data. This configuration was done as well to the Decision Tree.

```
Evaluating Random Forest Model
Accuracy of Random Forest: 82.09%
              precision    recall  f1-score   support

           0       0.63      0.89      0.74       778
           1       0.45      0.78      0.57       537
           2       0.71      0.47      0.56       690
           3       1.00      1.00      1.00       444
           4       1.00      0.88      0.94      1081
           5       0.85      1.00      0.92       693
           6       1.00      0.26      0.41       472
           7       1.00      1.00      1.00       859
           8       0.89      0.88      0.88       729
           9       1.00      0.83      0.91       780

    accuracy                           0.82      7063
   macro avg       0.85      0.80      0.79      7063
weighted avg       0.86      0.82      0.82      7063

Evaluating Decision Tree Model
Accuracy of Decision Tree: 79.14%
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       778
           1       1.00      1.00      1.00       537
           2       1.00      1.00      1.00       690
           3       0.36      1.00      0.53       444
           4       0.61      1.00      0.76      1081
           5       0.00      0.00      0.00       693
           6       1.00      1.00      1.00       472
           7       1.00      1.00      1.00       859
           8       1.00      1.00      1.00       729
           9       0.00      0.00      0.00       780

    accuracy                           0.79      7063
   macro avg       0.70      0.80      0.73      7063
weighted avg       0.69      0.79      0.72      7063
```

The KNN Model:

- I increased the default value (5) of n_neighbors to 20 because considering more neighbors of data points generally smoothens the decision boundaries/predictions of the model. However, it just lowered a very small percentage (from 99.90% to 99.04%), but it may still maintain high performance in clean data. When I tried to add noisy data in the training and test sets, the accuracy of the model dropped to 54.37%. This is because KNN is a distance-based algorithm wherein if I add noisy data points, it significantly affects the predictions because the algorithm might incorrectly assign a noisy data point to the wrong class based on its proximity, leading to lower classification accuracy. In short, the more I add noise, the less the model generalizes the actual patterns in the data since it relies upon incorrect data points. Therefore, it is normal for the KNN performance to drastically degrade when noisy data is introduced due to its sensitivity.

```
Accuracy of K-Nearest Neighbors: 99.04%
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       778
           1       0.99      0.99      0.99       537
           2       0.99      0.98      0.98       690
           3       0.99      0.98      0.99       444
           4       0.99      1.00      0.99      1081
           5       1.00      0.99      0.99       693
           6       0.97      0.98      0.98       472
           7       1.00      1.00      1.00       859
           8       1.00      0.99      0.99       729
           9       0.98      0.99      0.99       780

    accuracy                           0.99      7063
   macro avg       0.99      0.99      0.99      7063
weighted avg       0.99      0.99      0.99      7063

Evaluating K-Nearest Neighbors Model with Noise
Accuracy of KNN on noisy data: 54.37%
              precision    recall  f1-score   support

           0       0.56      0.93      0.70       778
           1       0.61      0.82      0.70       537
           2       0.54      0.51      0.53       690
           3       0.33      0.51      0.40       444
           4       0.58      0.39      0.47      1081
           5       0.51      0.67      0.58       693
           6       0.45      0.34      0.39       472
           7       0.58      0.34      0.43       859
           8       0.62      0.51      0.56       729
           9       0.60      0.49      0.54       780

    accuracy                           0.54      7063
   macro avg       0.54      0.55      0.53      7063
weighted avg       0.55      0.54      0.53      7063
```

Lastly, The Logistic Regression Model:

- The C parameter was increased to 500
- The max_iter parameter was reduced to 500

A large value of C prevents it from perfectly fitting the data, making it a more general model, similar to what was done to the Random Forest and Decision Tree.

```
Evaluating Logistic Regression Model
Accuracy of Logistic Regression: 89.71%
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       778
           1       1.00      1.00      1.00       537
           2       0.56      0.62      0.59       690
           3       0.94      0.78      0.85       444
           4       1.00      0.99      0.99      1081
           5       1.00      1.00      1.00       693
           6       0.82      0.73      0.77       472
           7       1.00      1.00      1.00       859
           8       1.00      1.00      1.00       729
           9       0.66      0.71      0.68       780

    accuracy                           0.90      7063
   macro avg       0.90      0.88      0.89      7063
weighted avg       0.90      0.90      0.90      7063
```

Summary of Results for the Models

Model	Accuracy	Error Rate	Precision	Recall	F-1 score	K-S Statistic	Gini Coefficient
Random Forest	82.09%	17.91%	0.85	0.80	0.82	1.00	1.00
Decision Tree	79.14%	20.86%	0.70	0.80	0.79	1.00	1.00
K-Nearest Neighbors	99.04%	0.96%	0.99	0.99	0.99	0.9981	0.9963
Logistic Regression	89.71%	10.29%	0.90	0.88	0.90	1.00	1.00

I then trained and evaluated the models' performance.

- The Random Forest model correctly predicted 82% of the test data while roughly around 18% of the predictions were incorrect. It achieved a perfect recall in classes 7 and 3 but was challenged with class 6 (0.26). All in all, it still has a high accuracy and is effective for classification tasks.
- The Decision Tree model recorded a slightly lower accuracy (79%) than the RF model. Some of the classes were classified perfectly, however, it struggled in others where it was unable to predict effectively. To conclude, the decision tree's performance may be skewed and may not be as reliable as other models.
- The KNN model showed an exceptionally high accuracy of 99%, and an extremely low error rate. All of its metrics are scored consistently high, implying that it has minimal misclassification and is considerably reliable.
- The Logistic Regression model also achieved high accuracy (90%). It generally demonstrated robust performance due to its weighted average F-1 score, despite having

low performance in certain classes. However, it may still struggle to accurately predict minority classes

Model Evaluation With PCA and Without PCA

```
Random Forest Model without PCA:
Evaluating Random Forest Model (PCA=False)
Accuracy: 82.09%
              precision    recall  f1-score   support

           0       0.63      0.89      0.74       778
           1       0.45      0.78      0.57       537
           2       0.71      0.47      0.56       690
           3       1.00      1.00      1.00       444
           4       1.00      0.88      0.94      1081
           5       0.85      1.00      0.92       693
           6       1.00      0.26      0.41       472
           7       1.00      1.00      1.00       859
           8       0.89      0.88      0.88       729
           9       1.00      0.83      0.91       780

    accuracy                           0.82      7063
   macro avg       0.85      0.80      0.79      7063
weighted avg       0.86      0.82      0.82      7063


Random Forest Model with PCA:
Evaluating Random Forest Model (PCA=True)
Accuracy: 46.38%
              precision    recall  f1-score   support

           0       0.81      0.97      0.88       778
           1       0.32      0.38      0.35       537
           2       0.56      0.42      0.48       690
           3       0.08      0.02      0.04       444
           4       0.59      0.15      0.23      1081
           5       0.24      0.28      0.26       693
           6       0.32      0.40      0.36       472
           7       0.93      0.50      0.65       859
           8       0.58      0.82      0.68       729
           9       0.26      0.56      0.36       780

    accuracy                           0.46      7063
   macro avg       0.47      0.45      0.43      7063
weighted avg       0.51      0.46      0.45      7063


Decision Tree Model without PCA:
Evaluating Decision Tree Model (PCA=False)
Accuracy: 79.14%
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       778
           1       1.00      1.00      1.00       537
           2       1.00      1.00      1.00       690
           3       0.36      1.00      0.53       444
           4       0.61      1.00      0.76      1081
           5       0.00      0.00      0.00       693
           6       1.00      1.00      1.00       472
           7       1.00      1.00      1.00       859
           8       1.00      1.00      1.00       729
           9       0.00      0.00      0.00       780

    accuracy                           0.79      7063
   macro avg       0.70      0.80      0.73      7063
weighted avg       0.69      0.79      0.72      7063
```

```
Decision Tree Model with PCA:
Evaluating Decision Tree Model (PCA=True)
Accuracy: 70.27%
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       778
           1       0.59      0.87      0.70       537
           2       0.69      0.48      0.56       690
           3       0.26      0.62      0.37       444
           4       0.81      0.38      0.52      1081
           5       0.76      0.66      0.70       693
           6       0.70      0.47      0.56       472
           7       1.00      0.99      1.00       859
           8       0.99      0.98      0.98       729
           9       0.48      0.59      0.53       780

    accuracy                           0.70      7063
   macro avg       0.73      0.70      0.69      7063
weighted avg       0.76      0.70      0.71      7063


K-Nearest Neighbors Model without PCA:
Evaluating K-Nearest Neighbors Model (PCA=False)
C:\Users\amand\anaconda3\Lib\site-packages\sklearn\metrics\_
t to 0.0 in labels with no predicted samples. Use `zero_divi
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
C:\Users\amand\anaconda3\Lib\site-packages\sklearn\metrics\_
t to 0.0 in labels with no predicted samples. Use `zero_divi
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
C:\Users\amand\anaconda3\Lib\site-packages\sklearn\metrics\_
t to 0.0 in labels with no predicted samples. Use `zero_divi
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
Accuracy: 99.04%
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       778
           1       0.99      0.99      0.99       537
           2       0.99      0.98      0.98       690
           3       0.99      0.98      0.99       444
           4       0.99      1.00      0.99      1081
           5       1.00      0.99      0.99       693
           6       0.97      0.98      0.98       472
           7       1.00      1.00      1.00       859
           8       1.00      0.99      0.99       729
           9       0.98      0.99      0.99       780

    accuracy                           0.99      7063
   macro avg       0.99      0.99      0.99      7063
weighted avg       0.99      0.99      0.99      7063
```

Observations:

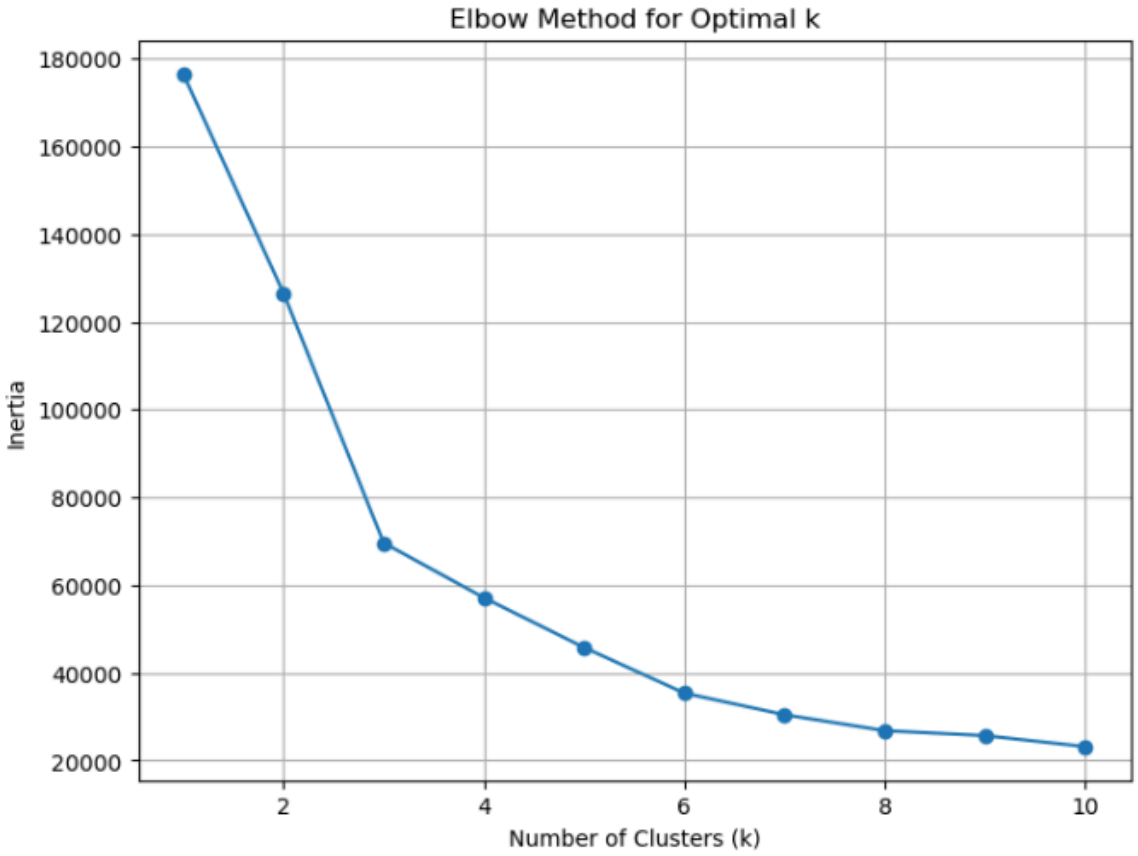
- The KNN model performs the strongest in both with or without PCA (both models are above 99%)
- Without PCA, Logistic Regression and Random Forest perform well with 89.71% and 82.09% respectively. While also Decision Tree performs average without PCA too (>79%)

- It is evident that with PCA, the accuracy scores dropped significantly except for the KNN model. This implies that PCA might be discarding important features that are critical for classification in these models. PCA simplifies the data by converting it into fewer dimensions. These models may depend heavily on feature interactions, which impact the predictions.

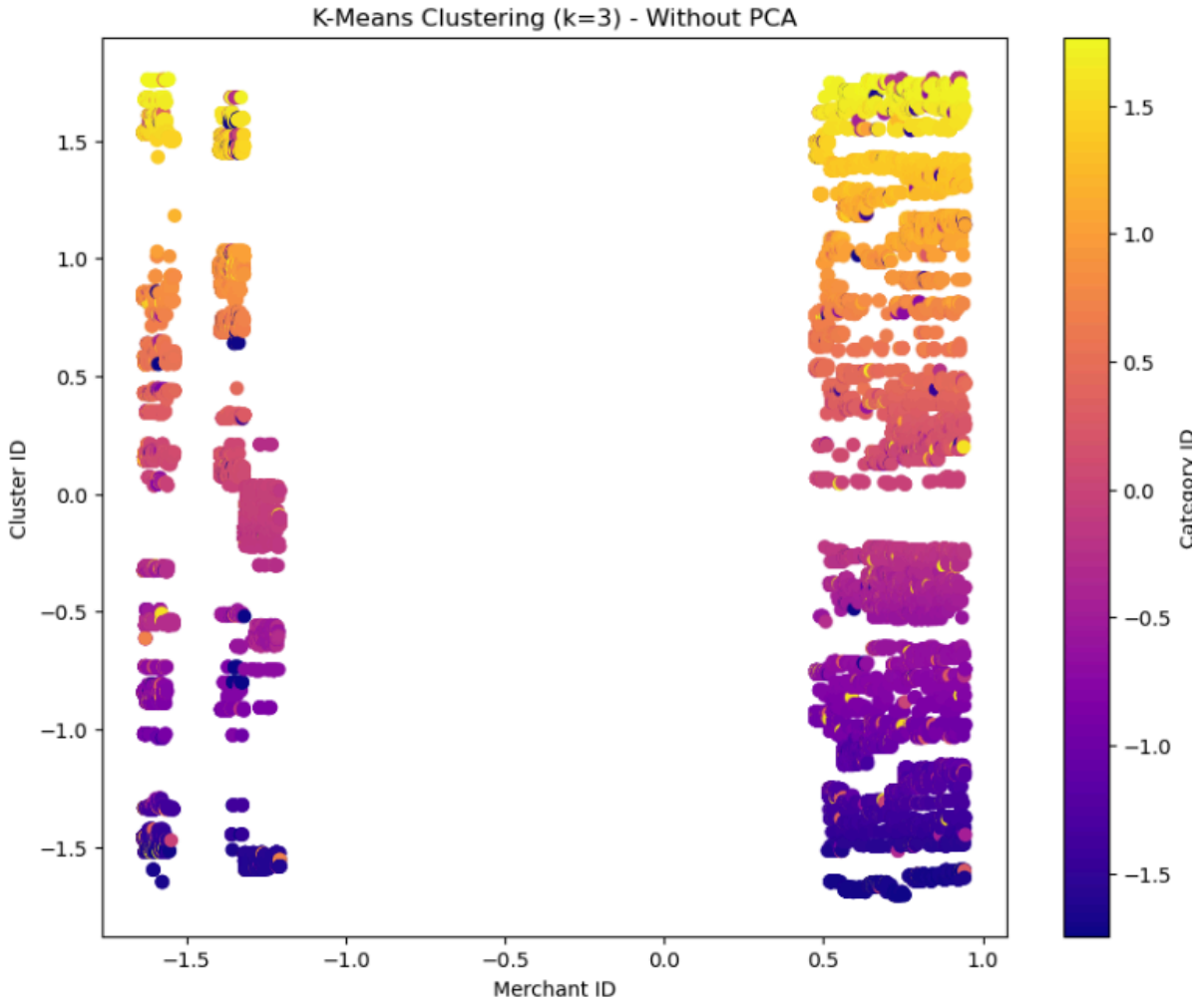
Conclusion:

I realized that the models fail to generalize new data and must have memorized the training set due to the vast capacity to learn the data from it. So adjusting some parameters can help the model generalize better and lead to more realistic accuracy. Increasing the strength of regularization and limiting the complexity of the model really helps with overfitting. I also applied SMOTE since it can help balance class distributions by creating synthetic examples of the minority class which makes the model have better performance and robust. KNN with PCA is the best model since it effectively captures local patterns in clustered data and reduces noise.

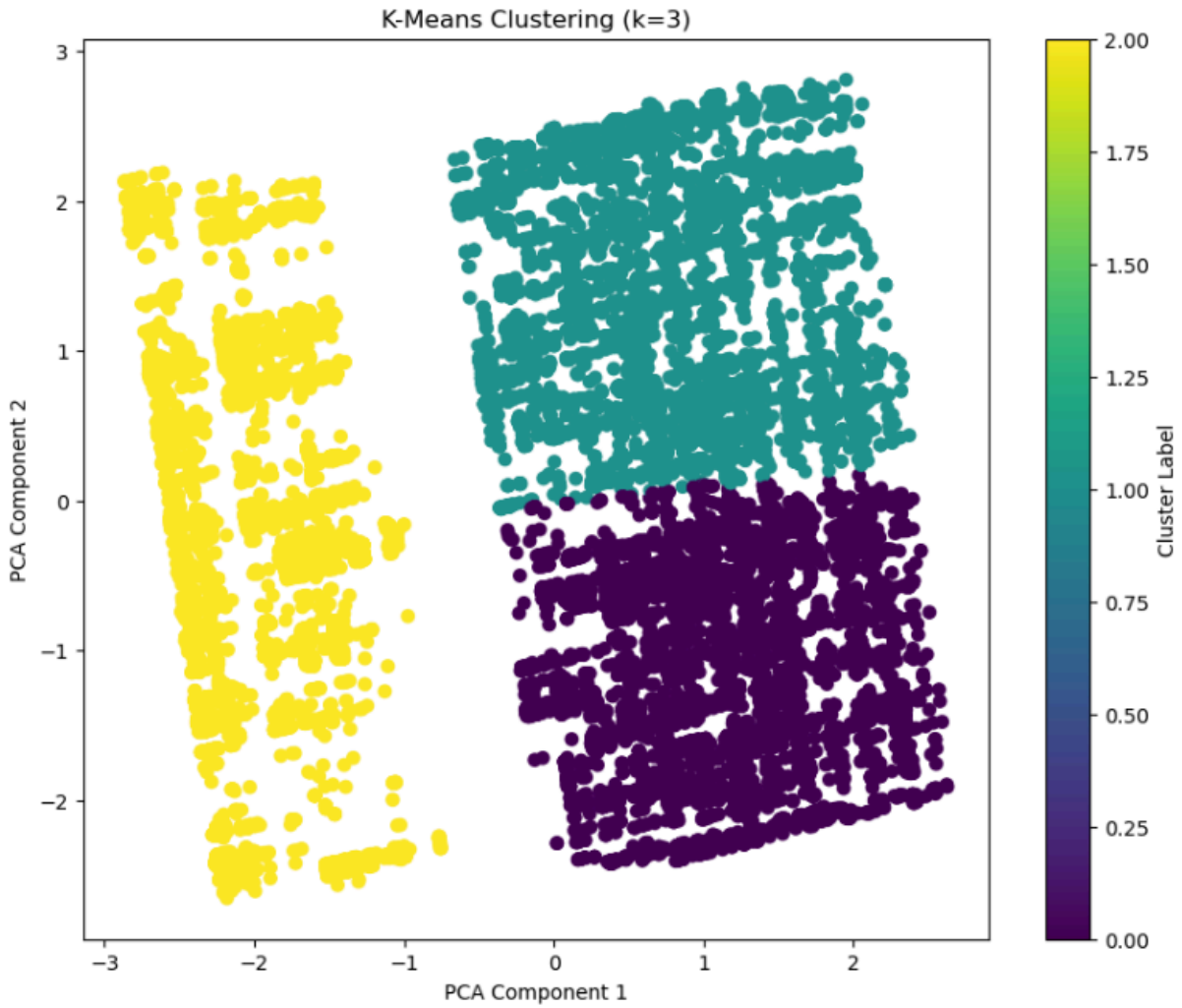
Clustering



The optimal k using the elbow method is 3. It appears that the elbow point in the graph is 3 clusters or k=3 since it sharply declines at this point or starts to form an elbow. To visualize the clusters, I did both visualizations with PCA and without PCA. There is a noticeable difference.

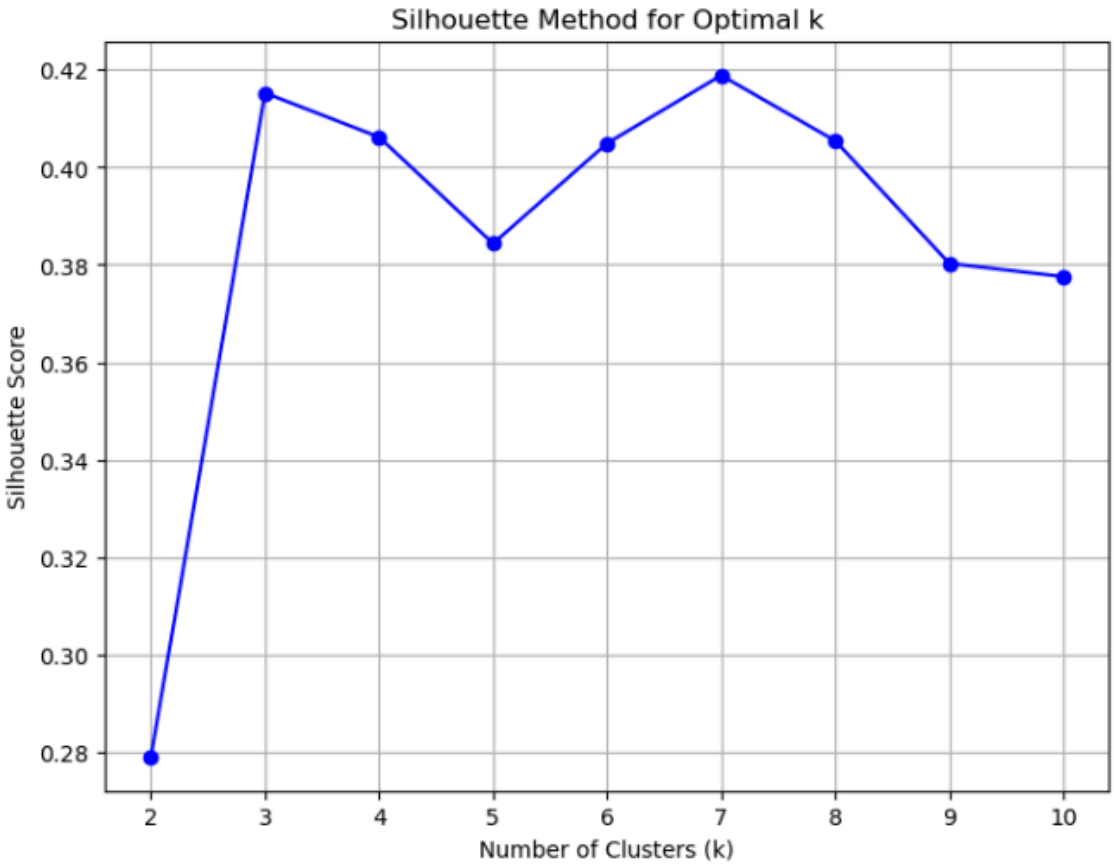


Without PCA, the features are directly plotted which means the data is still in its original higher-dimensional space. It includes noise or irrelevant features that make it complicated the separation clusters and is harder to visualize/analyse. Some features have more variance than others, but since there is no PCA, it is not prioritized. The complexity of the high-dimensional data is not reduced which makes the clusters overlap and harder to distinguish.

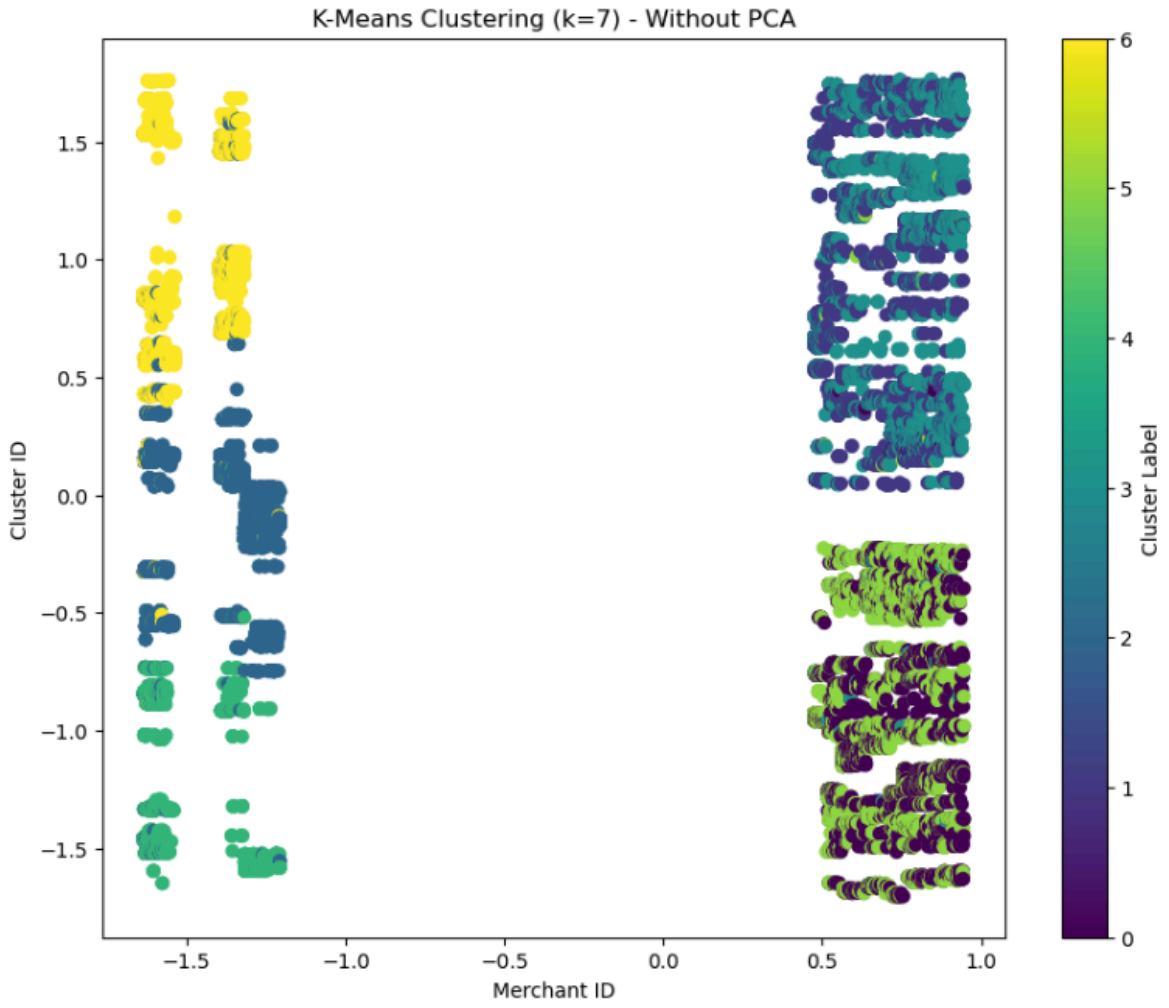


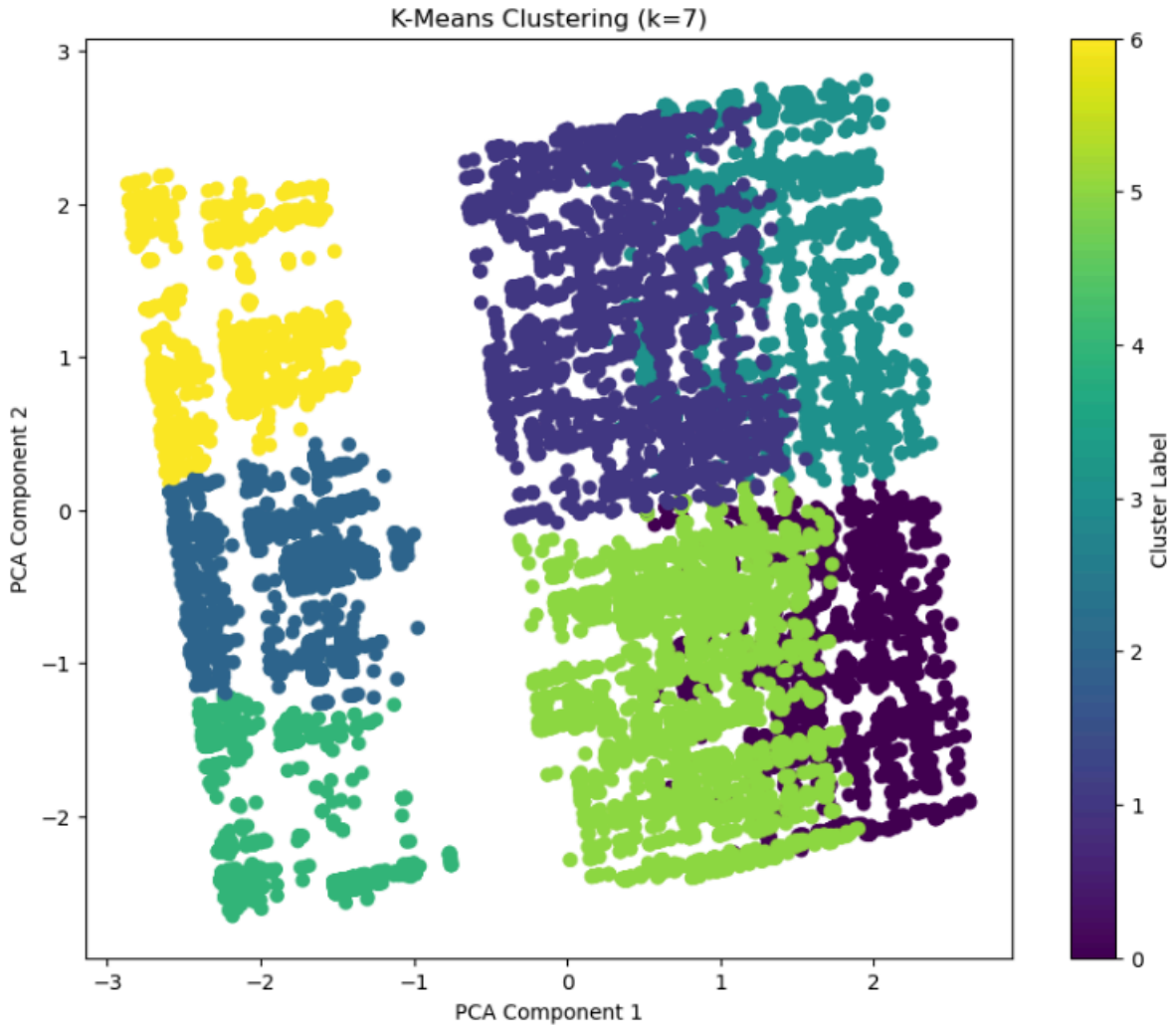
With PCA, it captures the most variance in the data by identifying the principal components, which filters out the irrelevant features and noise and only highlights the key patterns. It is evident that it provides a clearer separation between clusters, making it informative and visually appealing. In short, PCA presents a simplified version of the data by focusing on the important aspects and eliminating the irrelevant information.

I also used another type of clustering to find the optimal k, which is the Silhouette Method.



The results show that the optimal number of clusters based on the Silhouette method is k=7. The highest Silhouette score or peak is k=7 wherein it implies that 7 clusters is the best balance of cluster separation and chesion.


K-Means Clustering (k=7) - Without PCA
Cluster ID
Merchant ID
Cluster Label
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.5
−1.0
−0.5
0.0
0.5
1.0
6
5
4
3
2
1
0



I noticed it is generally much overlapped or harder to distinguish when PCA is not applied to the clusters on both Elbow and Silhouette methods. Meanwhile, PCA is much easier to interpret and has much more organized clusters.

Conclusion:

It is observed that the silhouette method has a different optimal k compared to the elbow method. The elbow method has an optimal k of 3 while the Silhouette method is 7. Both depends when to use since elbow method focuses on compactness and doesn't consider how well-seperated the clusters are. Meanwhile, the Silhouette Method evaluates both separation and cohesion. But, the elbow method seems more organized while the silhouette method may have closer clusters. K=3 might have a good balance between simplicity and representativeness since it provides a well-defined and distinct cluster. This is ideal for broader product and merchant categories. K=3 is better to use when developing more generalizable and manageable categories. K=7 reveals more detailed and refined insights that may allow for more precise segmentation, but it may

seem to overlap. Since k=3 is more organized and the data focuses more on improving product categorization, sales strategy, and product performance, this may be preferable. However, k=7 can provide more detail and is willing to handle the complexity for deeper analysis.

Presentation



Company — PriceRunner

- PriceRunner: Independent price comparison service
- Active in Sweden, UK, Denmark, Norway
- Over 5.8 million products, 15.4 million visits
- Dataset: UCI Machine Learning Repository
- Product Classification and Clustering dataset: 35,311 offers and 10 categories
- Suitable for clustering, classification, text-mining

Data Preparation

- I imported data analysis and machine learning libraries.
- There are **no missing values and duplicated rows** which creates a solid foundation for data analysis.
- There is a total of **35,311 rows and 7 columns**.
- I used the .**strip() method** to remove unnecessary space at the beginning of the columns.
- The **categorical columns** are: Product Title, Cluster Label, Category Label while **numerical columns** are: Product ID, Merchant ID, Cluster ID, and Category ID
- Identified inconsistencies and inaccuracies in product titles, so, i created a **4-gram visualization** for better multi-word pattern representation.
- **Verified frequent values** and addressed data standardization issues for accurate analysis.

```
Top 5 most frequent values:
Product Title
washing machine                          90
built in fully integrated dishwasher     35
frost free fridge freezer                34
washer dryer                             24
american fridge freezer                  22
Name: count, dtype: int64

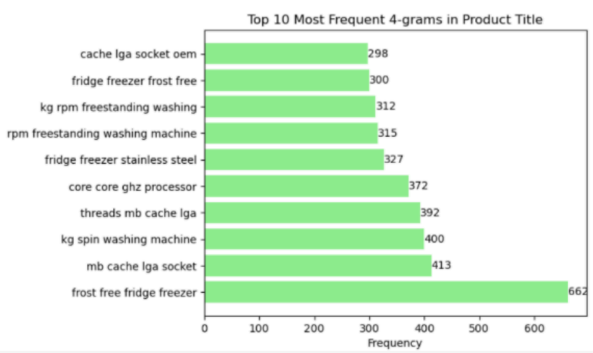
Top 5 most frequent values:
Cluster Label
Canon IXUS 185              27
Canon PowerShot SX730 HS    24
Samsung UE49NU7100          24
Apple iPhone 8 Plus 64GB    23
Samsung UE65NU7100          23
Name: count, dtype: int64

Top 5 most frequent values:
Category Label
Fridge Freezers     5501
Mobile Phones       4081
Washing Machines    4044
CPUs                3862
Fridges             3584
Name: count, dtype: int64
```

```
Product ID          0
Product Title       0
 Merchant ID        0
 Cluster ID         0
 Cluster Label      0
 Category ID        0
 Category Label     0
dtype: int64
```

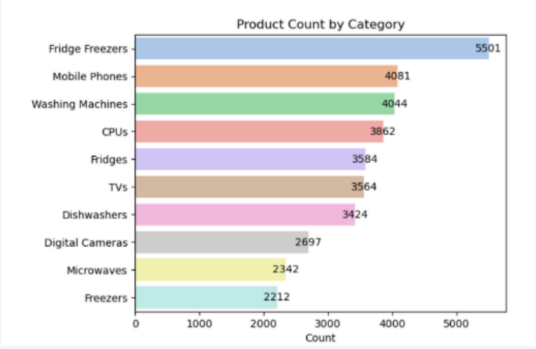
How can we improve product categorization, merchant performance, and overall sales strategy based on product titles, categories, clusters, and merchant behavior?

Visualisation #1: What are the most common 4-word combinations in product titles, and how can they assist in product categorization?



- 4-Grams improve product searchability and categorization.
- Facilitate product bundling, encourage upselling effectively.

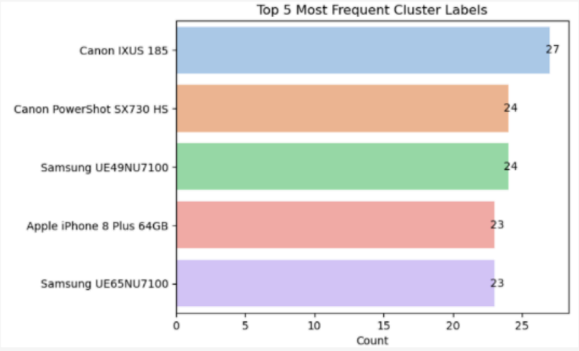
Visualization #2: Which product categories have the most products, and how can we use this for optimization?



- Focus on top product categories and seasons.
- Enhance product comparison features for detailed reviews.
- “Top Fridge Freezers of 2025” or “Popular Mobile Phones under £1000”

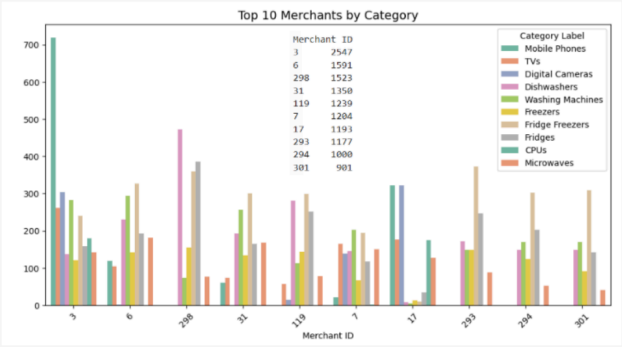
How can we improve product categorization, merchant performance, and overall sales strategy based on product titles, categories, clusters, and merchant behavior?

Visualization #3: What are the most frequent product clusters, and how can clustering boost sales?



- Cluster products by brand for great deals.
- Improve recommendations and targeted marketing campaigns.

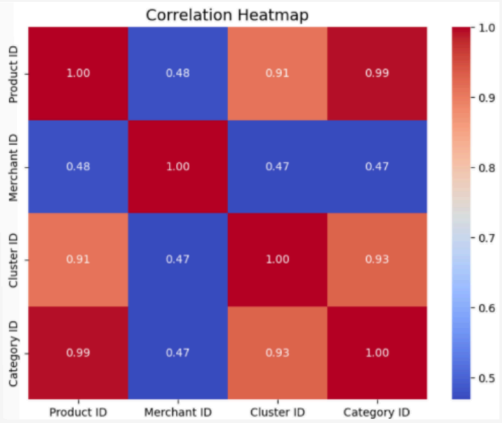
Visualization #4: How do merchants perform across different categories, and how can this information optimize performance?



- Focus on strongest categories for optimization.
- Enhance website navigation and merchant support.

How can we improve product categorization, merchant performance, and overall sales strategy based on product titles, categories, clusters, and merchant behavior?

Visualization #5: Which variables show strong relationships with each other?



- Strong correlation indicates systematic product categorization.
- Effective clustering groups similar products successfully.

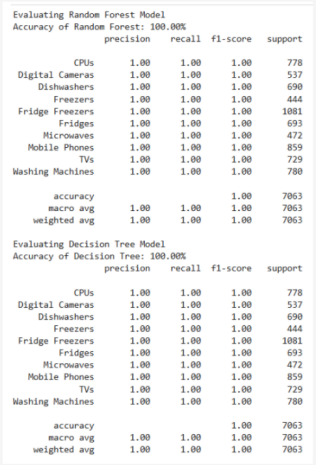
Conclusion

- 4-Grams for product titles effectively determines frequent term combinations.
- Maintains a strong correlation between Category ID and Product ID for product categorization.
- Encourage merchants to concentrate on their strongest categories .
- Maintain optimal inventory.
- Build strong relationships and promotional supports with merchants.
- Include detailed review ratings and side-by-side comparisons.
- Enhance recommendation systems through clustering techniques .
- Optimize website interface.

Classification

- **Target variable**: Category Label
- Models showed high to perfect accuracy (100%)
- Importance of **verifying model accuracy**, especially with easy-to-predict data.
- **Adjusted model settings** to achieve more realistic results and address high accuracy issues.
- Identified **overfitting** and **class/data imbalance** as the primary problems affecting model performance.
- Reduced n_estimators and max_depth, increased min_samples_split for Random Forest and Decision Tree; increased n_neighbors for KNN; adjusted C and max_iter for Logistic Regression.

Model	Accuracy	Error Rate	Precision	Recall	F-1 score
Random Forest	82.09%	17.91%	0.85	0.80	0.82
Decision Tree	79.14%	20.86%	0.70	0.80	0.79
K-Nearest Neighbors	99.04%	0.96%	0.99	0.99	0.99
Logistic Regression	89.71%	10.29%	0.90	0.88	0.90



```
Evaluating Random Forest Model
Accuracy of Random Forest: 100.00%
                   precision    recall  f1-score   support

             CPUs       1.00      1.00      1.00       778
  Digital Cameras       1.00      1.00      1.00       537
      Dishwashers       1.00      1.00      1.00       690
         Freezers       1.00      1.00      1.00       444
  Fridge Freezers       1.00      1.00      1.00      1081
         Fridges       1.00      1.00      1.00       693
      Microwaves       1.00      1.00      1.00       472
   Mobile Phones       1.00      1.00      1.00       859
             TVs       1.00      1.00      1.00       729
Washing Machines       1.00      1.00      1.00       780

        accuracy                           1.00      7063
       macro avg       1.00      1.00      1.00      7063
    weighted avg       1.00      1.00      1.00      7063

Evaluating Decision Tree Model
Accuracy of Decision Tree: 100.00%
                   precision    recall  f1-score   support

             CPUs       1.00      1.00      1.00       778
  Digital Cameras       1.00      1.00      1.00       537
      Dishwashers       1.00      1.00      1.00       690
         Freezers       1.00      1.00      1.00       444
  Fridge Freezers       1.00      1.00      1.00      1081
         Fridges       1.00      1.00      1.00       693
      Microwaves       1.00      1.00      1.00       472
   Mobile Phones       1.00      1.00      1.00       859
             TVs       1.00      1.00      1.00       729
Washing Machines       1.00      1.00      1.00       780

        accuracy                           1.00      7063
       macro avg       1.00      1.00      1.00      7063
    weighted avg       1.00      1.00      1.00      7063
```

Classification

Model Evaluation With PCA and Without PCA

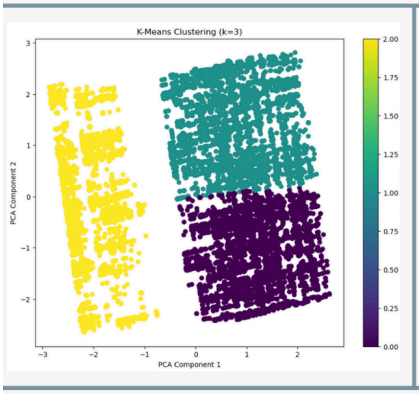
Model	PCA	Accuracy	Precision	Recall	F1-Score
Random Forest	False	82.09%	0.85	0.80	0.79
	True	46.38%	0.47	0.45	0.43
Decision Tree	False	79.14%	0.70	0.80	0.73
	True	70.27%	0.73	0.70	0.69
K-Nearest Neighbors	False	99.04%	0.99	0.99	0.99
	True	99.56%	0.99	0.99	0.99
Logistic Regression	False	89.71%	0.90	0.88	0.89
	True	71.97%	0.70	0.65	0.64

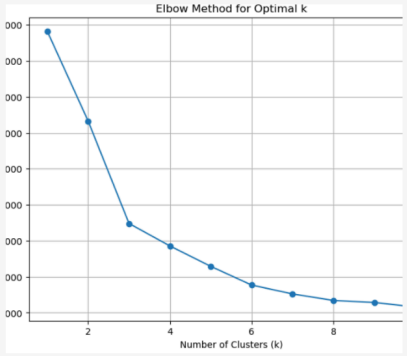
- KNN Model has the best performance with or without PCA (>99% accuracy).
- Without PCA: Logistic Regression (89.71%), Random Forest (82.09%), Decision Tree (>79%).
- With PCA: Accuracy drops for all models except KNN.
- PCA may discard crucial features where models rely on feature interactions

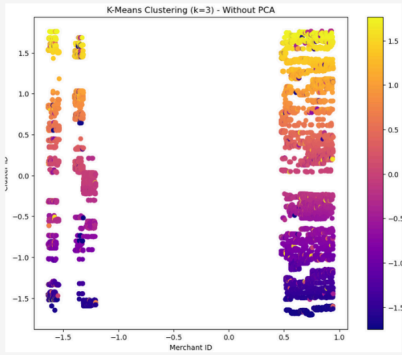
Conclusion

- Adjusting model parameters and applying SMOTE improved model performance and reduced overfitting.
- Achieved 82% accuracy but struggled with some classes. Decision Tree: 79% accuracy. While logistic regression performed at around 90%, facing challenges with minority classes.
- KNN is most reliable with over 99% accuracy, due to clear defined clusters.
- KNN with PCA is the best model since it effectively captures local patterns in clustered data and reduces noise.

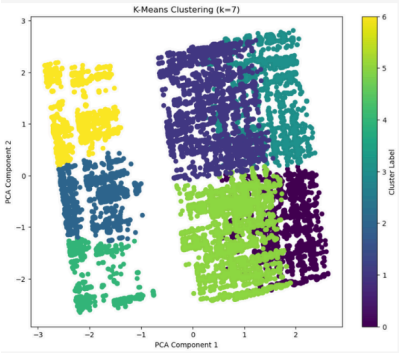
Clustering

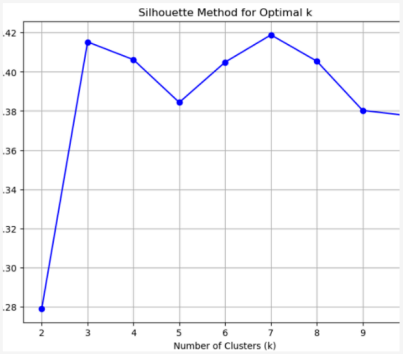


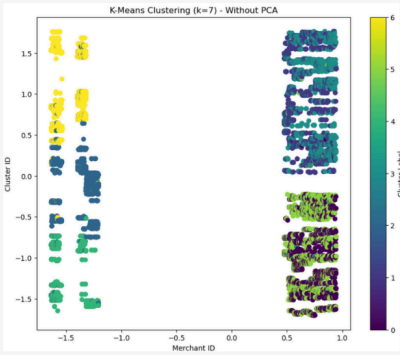




Clustering







Recommendations

- Price Runner may have a **systematic product categorization and recommendation systems** through clustering and classification to enhance both customer experience and searchability of products.
- Encourage merchants to concentrate on their **strongest categories and diversity in product offerings.**
- Create targeted marketing campaigns for **top product categories.**
- Use Random Forest for scenarios where high accuracy and **balanced performance** are important. Use Decision Trees where model **interpretability** like explaining decision to stakeholders. Use KNN where precision is highly imperative. Lastly, Logistic Regression serves as a **general-purpose model** or simplicity is preferred.
- Use K=3 (or Elbow Method) for **broader categorie**s like general product categorization, inventory management, and overall sales. Meanwhile, K=7 (or Silhouette Method) for **in-depth analysis** of product, merchant, and targeted marketing strategies.

Recommendations

- I recommend Price Runner to have more effective systematic product categorization and recommendation systems, through clustering and classification, to enhance both the searchability of products and customer experience.
- It is important to encourage merchants to concentrate on their best categories and diversify their product offerings.
- Create targeted marketing campaigns for top product categories like Fridge Freezers, Mobile Phones, and Washing Machines.
- The following classification models have their own distinct qualities, however, the KNN model is recommended due to its high precision. Other models can be used in certain situations:
    - Random Forest for scenarios where high accuracy and balanced performance are required.
    - Decision Trees where the model requires interpretability like explaining varying decisions to stakeholders
    - Logistic Regression for simplicity and general-purpose model.
- Use K=3 (or Elbow Method) for broader categories like general inventory management, product categorization, and overall sales. On the other hand, K=7 (or Silhouette Method) for in-depth analysis of product, merchant, and targeted marketing strategies.